

Reading Exercise 05 — GCC from a Distance

You will not read GCC. Nobody reads GCC. This exercise teaches you what to do instead.

What this exercise trains

GCC has millions of lines of code. It has been developed continuously since 1987. It supports dozens of source languages and target architectures. Reading it sequentially would take years.

This is not a failure of GCC or of the reader. Large codebases require a different reading strategy: not sequential comprehension, but spatial orientation. You learn where things are before you learn what they do. You navigate before you read.

This exercise is about navigation in the absence of comprehension.

The rule for this exercise

You may not spend more than 5 minutes in any single file. Set a timer.

Step 1 — The top-level map [5 minutes]

```
ls 02/gcc/gcc/
```

There are many subdirectories. Skim the list.

Write down:

- How many immediate subdirectories are there in 02/gcc/gcc/?
 - Name five that you can guess the purpose of from their names alone
 - Name one that surprises you
-

Step 2 — Count the source [3 minutes]

```
find 02/gcc/gcc -name "*.c" | wc -l  
find 02/gcc/gcc -name "*.c" | xargs wc -l 2>/dev/null | tail -1
```

Write down: the total number of `.c` files and the total line count. Compare this to TCC (which you now know intimately). What is the ratio?

Step 3 — Find the C parser [5 minutes]

GCC's C front end is in a specific file. Find it:

```
find 02/gcc/gcc -name "c-parser*"
wc -l 02/gcc/gcc/c-parser.c
```

Open `c-parser.c` for exactly 5 minutes. Look at the overall structure — the function names, the patterns, the scale.

Write down:

- How many lines is it?
 - Find the function that parses an `if` statement. What is it called? (Search for "if" in the function names: `cp_parser_if` or `c_parser_if`.)
 - Compare: TCC handles `if` in roughly 20 lines inside `block()`. How many lines does GCC use?
-

Step 4 — Find the optimizer [5 minutes]

GCC has an optimizer. TCC does not. The optimizer is one of GCC's defining architectural features. Find where it lives:

```
ls 02/gcc/gcc/ | grep -i "optim\|pass\|fold"
find 02/gcc/gcc -name "passes.c" -o -name "tree-pass.h" | head -5
```

Open `passes.c` for 5 minutes.

Write down:

- What is a "pass" in GCC's architecture?
 - Approximately how many optimization passes does GCC define or invoke? (Search for `pass_` in `passes.c`.)
 - How does this compare to TCC, which has zero optimization passes?
-

Step 5 — Find a comment you couldn't find in TCC [10 minutes]

GCC is old. It has accumulated comments that only appear in codebases with decades of history — comments about bugs fixed long ago, design decisions made before most of the current contributors were born, compatibility constraints from hardware that no longer exists.

Spend 10 minutes searching for such a comment. Try:

```
grep -rn "originally\\|historical\\|compat\\|quirk\\|workaround\\|regression" \
  02/gcc/gcc/c-parser.c 02/gcc/gcc/passes.c 02/gcc/gcc/fold-const.c | head
-30
```

Write down: the best comment you found — one that could not exist in a codebase as young as TCC. Copy it verbatim, with the file and line number.

Step 6 — Navigation summary [5 minutes]

You have spent roughly 40 minutes in a codebase with millions of lines. You have not read it. You have oriented yourself.

Write down:

- Where is the C parser? (file name)
- Where is the optimizer? (file name or directory)
- Where is the compiler driver — the entry point? (file name)
- One thing you know about GCC's architecture that you didn't know before

You now know where things are. That is enough for today.

Step 7 — The contrast [5 minutes]

You have now read TCC in some depth and navigated GCC from a distance.

Write down, in one paragraph: what is the most significant architectural difference between TCC and GCC as you understand them now? Not from the commentary documents — from what you have seen in the code itself.

Debrief

You cannot read GCC. Nobody can. But you can know where the C parser is, where the optimizer lives, where the entry point is. You can find a specific function when you need it. You can follow a specific

thread when you have a specific question.

That is what navigating a large codebase looks like. Not comprehension. Orientation. The ability to find your way.

You have it now. TCC is a territory you know. GCC is a territory you have visited briefly. Both are legible to you in a way they were not before this unit.

That is the goal of the whole unit: not to master the code, but to be able to read it — to stand inside it, find your bearings, and follow your curiosity wherever it leads.

The reading exercises are complete. Return to the problem sets for deeper work, or open the Unit 02 commentary documents to read alongside what you have found.